

Assignment 2

Where I Stand: ML, Deep Learning, AI & LLMs

1. Self-Rating

Area	Rating	In my own words
Machine Learning	A	I can take an ML project from start to finish on my own: cleaning and preparing data, engineering features, choosing a model, training it, evaluating it, and tuning it. I do this in Python with tools like Scikit-learn, XGBoost, and LightGBM.
Deep Learning	A	I build, train, and evaluate deep learning models on my own using TensorFlow/Keras and PyTorch. I've worked hands-on with neural networks, CNNs, and computer vision, and I'm comfortable with techniques like dropout, batch normalization, and regularization to get models performing well.
Artificial Intelligence	B	I've applied AI to real problems, including computer vision, classification, anomaly detection, and intelligent monitoring systems. My VISION project, an AI-based online exam proctoring system, shows I can pick the right approach and build it into a working end-to-end application.
Large Language Models	B	I understand how LLMs work, including transformers, embeddings, prompt engineering, vector databases, RAG, and inference, and I can build LLM-based application workflows. I'm still growing my experience with production-scale systems, and that's where I'm focusing next.

2. Why I Rated Myself an A in ML and Deep Learning

These two ratings come from both formal training and a lot of hands-on practice.

I completed my Data Science training at C-DAC (Centre for Development of Advanced Computing), a Government of India organization, and finished first in my course. Along the way I worked on several Machine Learning and Deep Learning projects, following each one through the whole process: preprocessing and exploratory analysis, then model building, training, evaluation, and optimization.

I worked with well-known datasets such as Iris and MNIST, as well as other classification and computer vision problems. That gave me real experience with a range of algorithms and techniques, not just the theory behind them.

I've put many of these projects on my GitHub, so you can read the code and see the work for yourself.

Between the training and my own projects, I'm confident that I can build, train, evaluate, and debug ML and Deep Learning models independently.

3. Why I Rated Myself a B in AI

My AI rating is backed by VISION (Visual Intelligence System for Invigilation and Online Monitoring) where I used AI, Deep Learning and Computer Vision to solve a real-world problem in examinations. VISION detects multiple faces, mobile phones and unusual gaze behavior while automatically capturing evidence when suspicious activity is detected. What I value most is that the project went beyond building models—I worked on the complete AI pipeline, from live camera input and detection to evidence collection and administrator monitoring.

I rated myself a B because while VISION gave me hands-on AI experience I still want to improve areas such, as behavioral analysis, model robustness and advanced detection techniques.

4. Why I Rated Myself a B in Large Language Models

My B rating in Large Language Models is supported by real-world experience developing Notes Snap, a web application that uses a vision-enabled LLM to look at whiteboard notes. The application takes pictures. Turns them into neat organized study notes. It also creates five self-test questions on its own. This project gave me experience working with multimodal LLMs, prompt engineering, image processing and making structured outputs. The B rating shows my understanding, in practice. Also means there is more to learn about fine-tuning, evaluation and putting large-scale LLMs into use.

2. Architecture Guide: Building an LLM-Powered Chatbot

What are the key architectural components needed to create a chatbot based on an LLM?

An LLM chatbot is not a model with a text box, in front. The model is the brain. A good LLM chatbot must also have a way to talk to users. It must remember the conversation. It must be able to look up facts that it was never trained on. Finally it needs guardrails that keep it safe and on topic.

A restaurant is a good way to picture it. The LLM is the chef, but you still need a front-of-house team, a well-stocked pantry, a notebook of what each guest ordered, and a manager who makes sure quality stays high. Figure 1 shows how those pieces fit together.

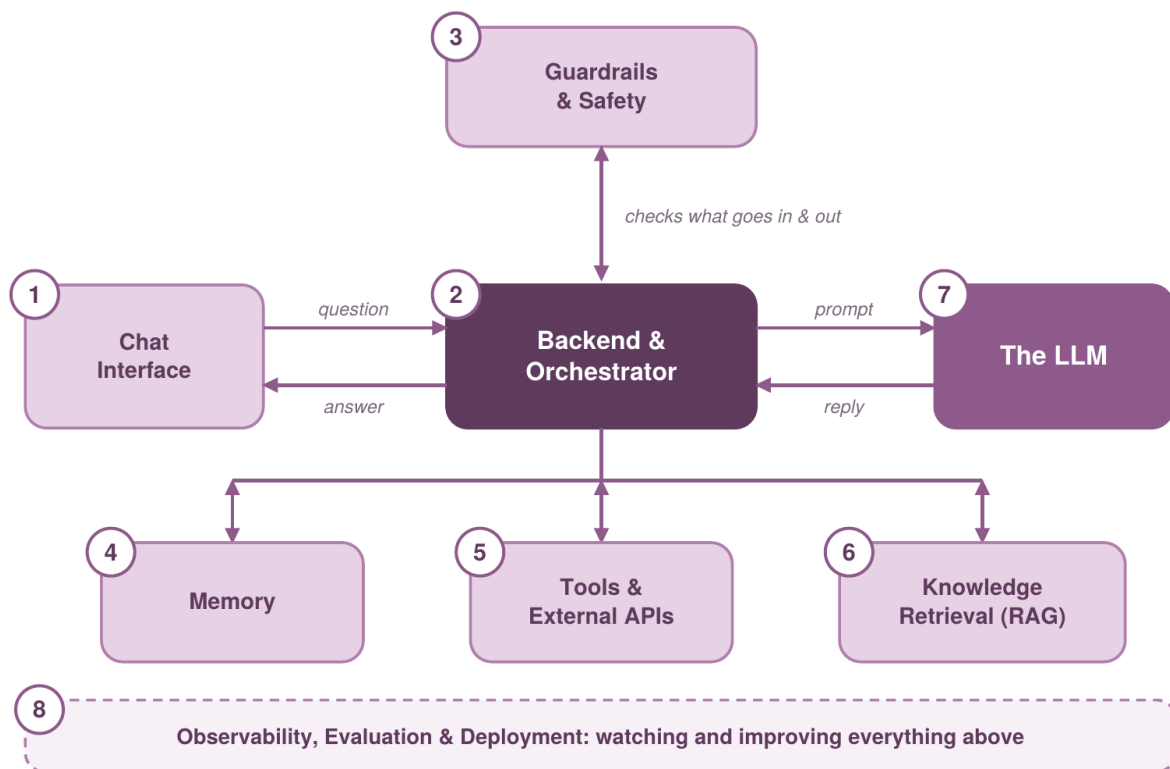


Figure 1. The orchestrator sits in the middle and coordinates everything else. Numbers match the list that follows.

Components

Each piece has a clear job. Here is what they do and the tools people commonly reach for.

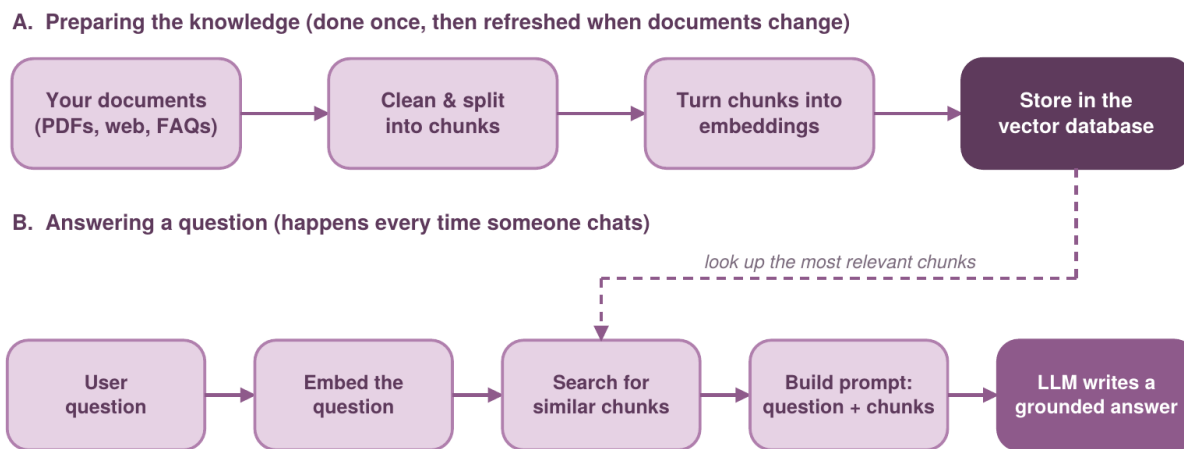
#	Component	What it does, in plain words
1	Chat Interface	This is where people actually talk to the bot: a web page, a mobile app, Slack, WhatsApp, or even a voice assistant. Its job is to feel quick and friendly, which is why good chat screens stream the answer word by word

#	Component	What it does, in plain words
		instead of leaving users staring at a blank box. Common choices: React, Streamlit, Gradio, Slack and WhatsApp APIs.
2	Backend & Orchestrator	Think of this as the traffic controller. It receives each message, checks who is asking, and decides what should happen next: look something up, call a tool, or simply answer. It also builds the final prompt by combining the system instructions, the chat history, any retrieved facts, and the new question. Common choices: FastAPI, Django, LangChain, LlamaIndex.
3	Guardrails & Safety	A quiet but important layer that checks what goes in and what comes out. It blocks prompt-injection tricks, hides personal data, filters harmful content, and keeps the bot on topic. Without it, a chatbot can be manipulated or say things you would never want a customer to see. Common choices: NeMo Guardrails, Guardrails AI, moderation APIs, custom rules.
4	Memory	An LLM forgets everything between calls, so the app has to remember for it. Short-term memory keeps the recent turns in the prompt, older turns get summarized to save space, and long-term memory stores things like user preferences or past sessions in a database. Common choices: Redis, PostgreSQL, rolling summaries.
5	Tools & External APIs	Sometimes the bot has to do something, not just say something: check an order, book a meeting, query a database, run a calculation. With function calling, the model asks the app to run an action and then uses the result in its reply. Common choices: function calling, REST APIs, SQL, MCP.
6	Knowledge Retrieval (RAG)	This lets the bot answer from your own documents instead of relying only on what it learned in training. Documents are split into chunks, turned into embeddings, and stored in a vector database. When a question arrives, the most relevant chunks are fetched and handed to the model, which cuts down hallucinations and keeps answers up to date. Common choices: FAISS, Chroma, Pinecone, Weaviate, pgvector.
7	The LLM	The brain of the whole system. You can call a hosted model through an API, which is quick to start and pay-as-you-go, or run an open-source model yourself, which gives more control and privacy but means managing your own infrastructure. The right pick depends on quality, cost, speed, and how sensitive your data is. Common choices: Claude, GPT, Gemini, Llama, Mistral.
8	Observability, Evaluation & Deployment	Once the bot is live, you need to see how it is really doing. Logging conversations, tracking cost and response time, collecting thumbs up or down, and regularly testing answer quality all help you catch problems early. Caching, containers, and autoscaling keep it fast and reliable as more people use it. Common choices: LangSmith, Langfuse, Prometheus, Docker, Kubernetes.

How RAG gives the bot real knowledge

RAG (Retrieval-Augmented Generation) deserves to be looked at closely because it is what makes many business chatbots truly helpful and trustworthy. RAG operates in two steps. First the knowledge base is set up by breaking documents into pieces. Each piece is then turned into an embedding—a set of numbers that shows what it means—and placed in a vector database.

Every time a user asks a question that question is also turned into an embedding. RAG then looks through the vector database to find the pieces that match the question the most. Those selected pieces are given to the LLM along with the question, which helps the LLM create an answer using the right information, from the knowledge base.



The model answers from your documents, so replies stay current and are far less likely to be made up.

Figure 2. The two stages of RAG: preparing the knowledge, then answering with it.

What happens when you send one message

Putting it all together, this is the journey a single message takes, usually in about a second or two.

1. **You send a message.** The chat interface passes it to the backend.
2. **Safety check.** Input guardrails look for anything harmful, private, or suspicious.
3. **Remember the conversation.** The orchestrator pulls in recent history and any saved preferences.
4. **Look things up.** If the question needs facts, the retriever fetches the best-matching document chunks.
5. **Build the prompt.** Instructions, history, retrieved context, and your question are combined. If an action is needed, the model asks for a tool and the app runs it.
6. **The LLM answers.** The reply streams back token by token.
7. **Final check and log.** Output guardrails review it, you see it on screen, and the exchange is logged for evaluation.

A common mistake is jumping straight to training. I would start at the left of Figure 3 and only move right when the simpler step is not enough. Good prompts solve more problems than people expect, RAG covers most knowledge needs, and fine-tuning is best saved for teaching a specific tone or behavior.

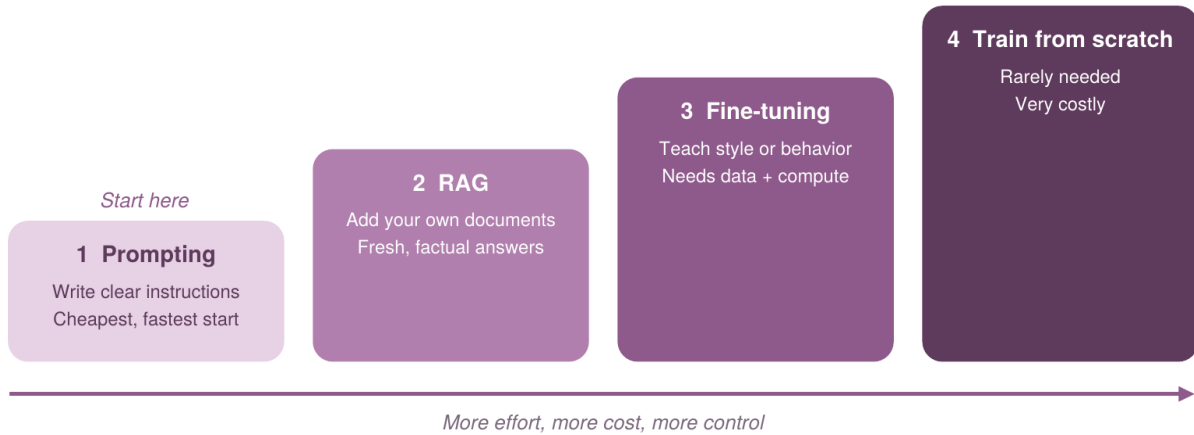


Figure 3. Start simple and add effort only when you need it.

A great LLM chatbot is a system, not just a model: a friendly interface, a smart orchestrator, safe guardrails, good memory, reliable knowledge, and constant monitoring, all wrapped around the LLM.

3. Technical Guide: Vector Databases, Explained

Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem), which one would you choose, and why?

Imagine a database like a filing cabinet. You have to know the label, keyword or ID to get the information you need. A vector database is different. It acts like a smart assistant. Of searching only for exact words it understands what you mean and the situation you are in. It locates information that's similar, in meaning even if the words are not the same.

I find that this works because modern AI models can turn text, images or audio into embeddings which're long lists of numbers that keep similar meanings close together. A vector database stores millions of these embeddings. Can locate the ones nearest to your query in just a few milliseconds. The vector database is the engine, behind the Knowledge Retrieval (RAG) component mentioned earlier. It is why a chatbot can answer from your own documents.



Figure 4. Meaning as geography: the question lands near the refund articles, so those are what comes back.

How the search actually works

Comparing your question to every stored vector would take too long if you have millions of them. That's why the database creates an index in advance. It then uses nearest-neighbour search or ANN to quickly jump to the most likely areas. On top of that most systems allow you to add fields like language or product type. This means you can search by meaning and by rules, at the time.

The key ideas, in plain words

Concept	What it means
Embedding	A list of numbers (often a few hundred to a few thousand) that captures what a piece of text, an image, or a clip of audio means. Similar things get similar numbers.
Similarity metric	How closeness is measured. Cosine similarity, which compares the angle between two vectors, is the usual choice for text. Dot product and Euclidean distance are also common.
Approximate nearest neighbour (ANN)	Checking every vector one by one is far too slow at scale, so databases use smart shortcuts that find almost-the-best matches in milliseconds. You give up a sliver of accuracy for a huge gain in speed.
HNSW index	A layered road map of vectors. The search starts on the highways, then zooms into local streets near your answer. Fast and accurate, but it likes memory.
IVF index	Sorts vectors into clusters, like aisles in a library, and only searches the aisles most likely to hold your answer. Lighter on memory, a little less precise.
Quantization	Compresses vectors, for example by storing them with fewer bits, to save memory. Usually costs a small amount of accuracy.
Metadata filtering	Narrowing the search with ordinary fields such as language, product, or date, so you only compare items that are actually relevant.
Hybrid search	Combining search by meaning with plain keyword search, so both paraphrased questions and exact terms like error codes work.

A tour of the popular options

There is no single best vector database, only the best fit. Here is how I see the main contenders.

Option	Shines when	Watch out for
FAISS	Fast experiments and research. A library you embed in your own code.	It is a library, not a full database, so storage management, filtering, and APIs are up to you.
Chroma	Prototypes, notebooks, and small apps. Very easy to start with.	Think carefully before leaning on it for large production workloads.
pgvector	Teams already on Postgres who want everything in one place.	Very large collections or heavy filtered search can need extra tuning.
Pinecone	A fully managed service with zero servers to look after.	Managed-only, so no self-hosting, and costs rise as vectors and traffic grow.

Option	Shines when	Watch out for
Qdrant	An open-source engine with strong filtering and flexible deployment.	A smaller ecosystem of ready-made integrations than the biggest names.
Weaviate	Built-in hybrid search and optional vectorizer modules.	Keeps its index in memory by default, so RAM needs can grow quickly.
Milvus / Zilliz	Massive scale, from hundreds of millions to billions of vectors.	More moving parts to run, and overkill for smaller projects.

A hypothetical problem to choose for

To make the choice concrete, imagine a support assistant for a mid-sized online electronics store. Customers ask questions in a chat window, and the bot answers from the company’s own help content using RAG.

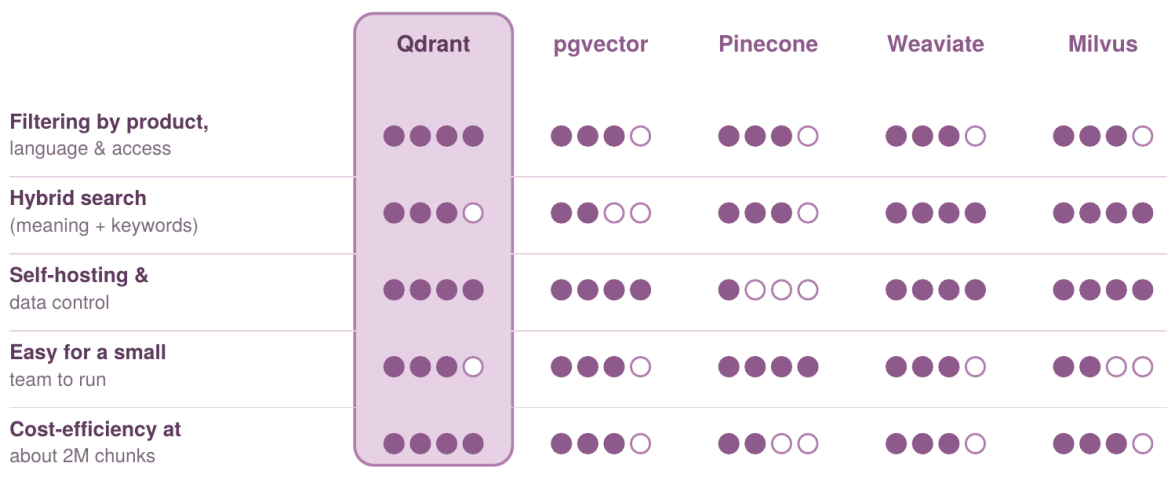
Requirement	Detail
Data	About 300,000 help articles, manuals, and resolved tickets, which become roughly 2 million chunks.
Traffic	Up to about 50 searches per second at peak, with answers expected in under a second.
Filters	Every search must respect product line, language (English or Hindi), and access level (public or agent-only).
Exact terms	Customers paste error codes and model numbers, so keyword matching matters as much as meaning.
Privacy	Customer data has to stay inside the company’s own cloud region.
Team and budget	Three engineers, no dedicated database admin, a tight budget, and the main app data lives in MongoDB, not Postgres.

My pick: Qdrant

For this scenario I would choose Qdrant. The deciding factors are the heavy filtering, the need for keyword matching alongside meaning, the privacy requirement, and a small team on a tight budget. Here is why it fits:

1. **Filtering that does not hurt search quality.** Qdrant applies filters while it searches, not afterwards. So “English, public articles for this product” narrows the results without leaving the user with a handful of weak matches.
2. **Meaning and keywords together.** It can store dense vectors (meaning) and sparse vectors (keywords) and combine the results. A customer typing an error code and another describing the same problem in their own words should both land on the right article.

3. **Affordable at this size.** Quantization and storage options that save memory help keep things efficient. This means that around 2 million chunks can run on machines instead of needing a large and costly cluster.
4. **Open source, so the data stays home.** It is Apache 2.0 licensed and runs on the company's own infrastructure, which satisfies the privacy rule and avoids lock-in. If the team later wants less to manage, a managed cloud version exists.
5. **Friendly for a small team.** It runs as a simple service that starts with a single Docker command and can grow into a cluster later, with REST and gRPC APIs and a solid Python client.



More filled dots = better fit for this scenario. My own judgement, not a benchmark.

Figure 5. How the main contenders stack up for this scenario.

What I would be giving up

Qdrant has a smaller ecosystem of ready-made integrations than the biggest names, and Weaviate's built-in vectorizers would have saved a little glue code. Those are real trade-offs, but for this scenario they do not outweigh the filtering and cost advantages.

If the situation were different

My answer would change depending on the problem. If the same company ran everything on Postgres and had a few million chunks I would definitely go with pgvector and not add a new system. If no one wanted to handle servers Pinecone would be the choice. If the data expanded to hundreds of millions of vectors Milvus would be worth the work. Figure 6 shows the way I

would make the decision.

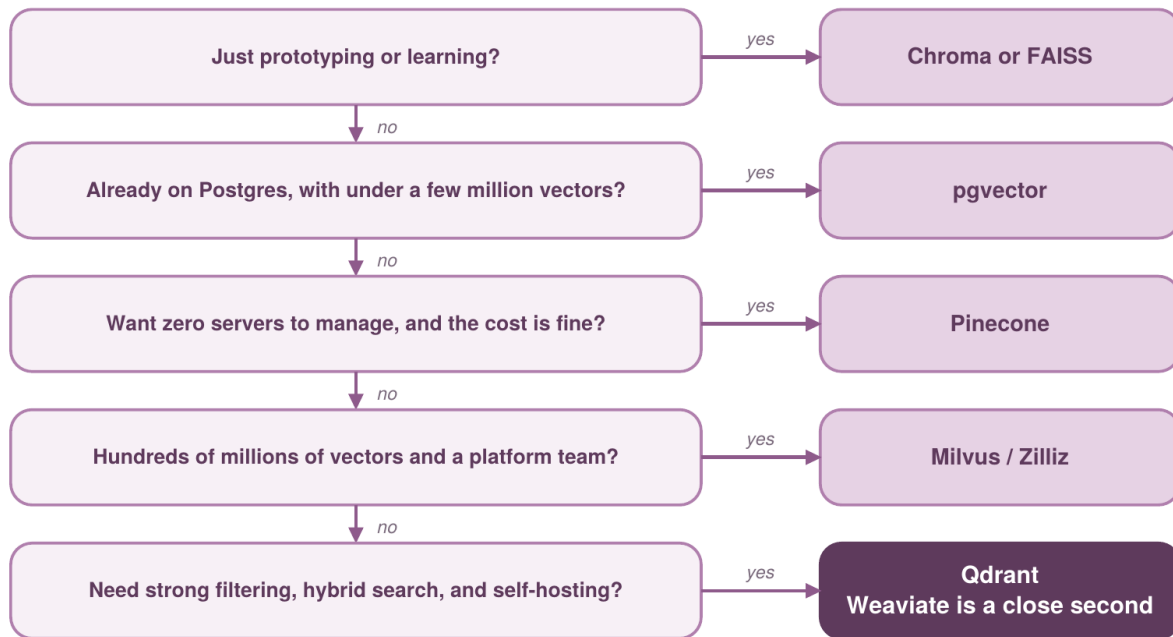


Figure 6. A quick decision guide, read from top to bottom.

How I would double-check the choice

Before making a decision I would run a test using a sample of actual support questions. I would check how well Qdrant, Weaviate and pgvector perform, in terms of recall .I would also look at percentile latency and monthly cost. These vector databases evolve fast so I would go back. Read each product’s latest documentation right before choosing..